NRLC-07: Logs, Tags & Drop Rules

> **Series:** NRLC — New Relic to Dynatrace Migration Deep Dives | **Notebook:** 7 of 9 | **Created:** April 2026 | **Last Updated:** 10/31/2026

Overview

The cost-optimization layer of the migration. NR's flat retention and ingest-time drop rules become DT's tiered Grail buckets, OpenPipeline filters, and entity tags. This deep dive covers log forwarding patterns, drop rule conversion, log parsing rule (Grok→DPL) translation, and tag taxonomy migration.

Phase 17 + 18 + 23 + 24 data-side coverage (post-2026-04-15): the engine now auto-converts a broad surface of data-layer configuration beyond drop/parse/tag:

Capability	Transformer	Phase
PII / PAN log obfuscation (7 presets + regex)	`log_obfuscation_transformer`	17
Custom event ingest (bizevent CloudEvent payloads)	`custom_event_ingest_transformer`	17
NR Log Live Archive → Grail bucket + egress (S3/GCS/Azure Blob)	`log_archive_transformer`	24
Metric normalization (rename / aggregate / drop processors)	`metric_normalization_transformer`	24
Database monitoring (10 engines: MySQL / Postgres / MSSQL / Oracle / MongoDB / Redis / Cassandra / MariaDB / DB2 / HANA)	`database_monitoring_transformer`	24

Capability	Transformer	Phase
On-host integrations (12 techs: NGINX / HAProxy / Kafka / RabbitMQ / Elasticsearch / Memcached / Couchbase / Consul / Apache / etcd / Varnish / Zookeeper)	`on_host_integration_transformer`	24
Security Signals / IAST	`security_signals_transformer`	24
Custom entities (NR entity platform → DT custom-device API)	`custom_entity_transformer`	24
Cloud integrations (AWS 16 / Azure 8 / GCP 8)	`cloud_integration_transformer`	18
Kubernetes → DynaKube	`kubernetes_transformer`	18
Prometheus ingestion	`prometheus_transformer`	18
OTel metrics / OTel collector (traces + metrics + logs + 5 processors)	`otel_metrics_transformer` (23) + `otel_collector_transformer` (24)	23/24
StatsD on ActiveGate	`statsd_transformer`	23
CloudWatch Metric Streams (Firehose)	`cloudwatch_metric_streams_transformer`	23
AI Monitoring (model registry + inference NRQL→DQL)	`ai_monitoring_transformer`	18
NPM (SNMP + NetFlow, secrets redacted)	`npm_transformer`	18
Vulnerabilities + per-CVE muting	`vulnerability_transformer`	18

See §4, §6, §15 and §16 of the coverage matrix (migration utilities GitHub) for the complete row-by-row mapping.

Table of Contents

1. Log Forwarding Patterns
2. Drop Rules → OpenPipeline Filters
3. Log Parsing — Grok → DPL
4. Tag Taxonomy Migration
5. Cost Optimization Patterns
6. Validation Queries

Prerequisites

Requirement	Details
Audience	Platform engineers, FinOps stakeholders
Standalone	This notebook is self-contained for logs/tags/drops migration. No required prerequisite reading.
Optional depth	NRLC-08 (validation), NRLC-09 (toolchain)
Recommended companion	ORGNZ-02 / ORGNZ-99 (Grail bucket strategy this maps onto)
Tooling	`Dynatrace-NewRelic`'s `LogParsingTransformer`, `DropRuleTransformer`, `TagTransformer`

Embedded Translation Context — Log Filter & Parsing Patterns

Drop rules translate from NRQL to an OpenPipeline matching condition written in DQL; parsing rules translate from Grok to DPL.

Drop rule (NRQL filter → OpenPipeline drop processor)

```
-- NRQL
DELETE FROM Log WHERE message LIKE '%health%' AND service = 'frontend'
```

```
# OpenPipeline "Drop record" processor — matching condition (DQL)
matchesValue(content, "*health*") and k8s.workload.name == "frontend"
```

The NR `service` attribute has no fixed Dynatrace counterpart on logs — map it to whatever attribute your logs actually carry (check with the coverage query in §6). On the validation tenant `service.name` was set on about 2% of logs and `k8s.workload.name` on about 22% (10/05/2026); a rule written against an attribute your logs lack matches nothing and drops nothing. Write substring tests as `matchesValue(field, "*text*")` — OPLOGS-03 records that matching conditions reject `contains()`.

Parsing rule (Grok → DPL)

```
-- Grok
%{IPORHOST:client_ip} - %{NUMBER:response_ms} %{TIMESTAMP_ISO8601:ts}
```

```
NSPACE:client_ip ' - ' DOUBLE:response_ms ' ' TIMESTAMP('yyyy-MM-
ddTHH:mm:ss.SSSZ'):ts
```

Tested with `data record(content = "…") | parse content, "…"` on `10.0.0.1 - 123 2026-10-05T12:00:00.123Z`, `10.0.0.1 - 123.5 2026-10-05T12:00:00.004Z` and `api.example.com - 123 2026-10-05T12:00:00.123Z` — all three parse (10/05/2026). The literal translation `IPADDR:client_ip ' - ' INT:response_ms ' ' TIMESTAMP('yyyy-MM-dd''T''HH:mm:ss.SSS'):ts` returns null on all three: `IPADDR` rejects host names (Grok `IPORHOST` accepts them), `INT` rejects decimals (Grok `NUMBER` accepts them), and the timestamp format does not cover the trailing `Z`.

Tag rule (NR entity tag → OpenPipeline enrichment)

```
# OpenPipeline enrichment that adds environment attribute to ingested logs
matcher: matchesValue(k8s.namespace.name, "*prod*")
field: environment
value: prod
```

Translation Confidence

Pattern	Confidence	Notes
Simple `WHERE` drop rule	HIGH	DQL matching condition on a Drop record processor — once the NR attribute is mapped to a field your logs carry
Drop rule with OR/AND/NOT	HIGH	Boolean operators map directly
Drop rule with regex	MEDIUM	Verify DPL regex equivalent
Standard Grok primitives	HIGH	Only with the right DPL matcher: `IPORHOST` → `NSPACE`, `NUMBER` → `DOUBLE`, `WORD` → `WORD` (see §3)
Custom Grok library	MEDIUM	May need DPL pattern reformulation
Tag rule on entity attribute	HIGH	Becomes OpenPipeline enrichment

1. Log Forwarding Patterns

NR's primary log ingest paths:

Source	NR Endpoint	DT Equivalent
Filebeat / Fluent Bit	NR Log API	OneAgent log ingest or Fluent Bit → OpenPipeline
Lambda log forwarder	NR Lambda extension	Dynatrace Lambda extension or CloudWatch → Firehose → OpenPipeline
K8s log integration	NR Kubernetes integration	DynaKube + log monitoring
Syslog	NR Syslog forwarder	OpenPipeline syslog ingest endpoint
HTTP/JSON	NR Log API	DT Generic Log Ingest API

Migration: reconfigure each forwarder to point at DT. Run dual-shipping (both NR and DT receive) for the dual-run period; cut NR after volume confirmation.

2. Drop Rules → OpenPipeline Filters

NR drop rules filter at ingest. DT's equivalent is an OpenPipeline **Drop record** processor, whose matching condition is written in DQL (not DPL — DPL is the parsing language).

NR drop rule (NRQL filter):

```
DELETE FROM Log WHERE message LIKE '%health%' AND service = 'frontend'
```

Equivalent OpenPipeline rule (Drop record processor, DQL matching condition):

```
# OpenPipeline "Drop record" processor — matching condition (DQL)
matchesValue(content, "*health*") and k8s.workload.name == "frontend"
```

Map NR `service` to an attribute your Dynatrace logs actually carry — `service.name` is often absent on OneAgent-collected logs (check with the coverage query in §6).

The `DropRuleTransformer` translates each NR drop rule to an OpenPipeline filter rule, preserving:
- the filter expression (NRQL → DQL matching condition)
- the source identification (NR data type → DT data table)
- the rule's enabled/disabled state

Translation is HIGH confidence for simple `WHERE` clauses; MEDIUM for clauses with regex or nested boolean logic.

3. Log Parsing — Grok → DPL

NR uses Grok patterns for log parsing; DT uses DPL (Dynatrace Pattern Language).

Both are pattern languages but with different syntax. The `LogParsingTransformer` handles common Grok primitives:

Grok	DPL
`%{IPORHOST:client_ip}`	`NSPACE:client_ip` (or `IPADDR:client_ip` when only IP addresses occur — `IPADDR` rejects host names)
`%{NUMBER:duration_ms}`	`DOUBLE:duration_ms` (`INT` rejects decimals)

Grok	DPL
`%{TIMESTAMP_ISO8601:timestamp}`	`TIMESTAMP('yyyy-MM-ddTHH:mm:ss.SSSZ'):timestamp` for fractional seconds; `ISO8601:timestamp` matches `2026-10-05T12:00:00Z` but returned null on `2026-10-05T12:00:00.123Z`
`%{WORD:method}`	`WORD:method`
`%{DATA:rest}`	`LD:rest`
`%{GREEDYDATA:everything}`	`LD:everything`

Conversion confidence:

- HIGH for patterns built from common Grok aliases
- MEDIUM for patterns with custom Grok library imports
- LOW for patterns relying on Grok's regex-anywhere capability (DPL is more positional)

DPL parses are part of OpenPipeline; once converted, they apply at ingest before data lands in the bucket.

DPL does not backtrack the way Grok's regex engine does, so a literal Grok-to-DPL substitution can parse nothing: `LD:m LD:p ' ' INT:s` returns null on `GET /api/cart 200`, while `WORD:m ' ' LD:p ' ' INT:s` parses it (10/05/2026). Test every converted pattern against a sample line with `data record(content = "<sample>") | parse content, "<pattern>"` before it goes into a pipeline. FAQ-15 covers DPL matching semantics in full.

4. Tag Taxonomy Migration

NR has account-wide tags applied to entities. DT has both **entity tags** (applied to discovered entities) and **Grail dimensions** (applied to ingested data).

NR Tag Type	Gen3 DT Equivalent
Entity tag (e.g., `env:prod` on a host)	OpenPipeline enrichment that adds the same attribute to incoming data (e.g., `environment = "prod"`)
Workload tag	OpenPipeline enrichment emitting `workload.name = "<value>"` + optional bucket routing

NR Tag Type	Gen3 DT Equivalent
Account-level tag	OpenPipeline enrichment scoped by ingest source / bucket

(Note: Gen2 entity tags and auto-tagging rules still exist, but the canonical Gen3 pattern enriches data at ingest so the attribute is queryable in DQL and usable in IAM bucket conditions.)

Recommended approach: convert NR tags to **OpenPipeline enrichment rules**. New ingested data automatically receives the attribute as it lands in Grail, so DQL queries and IAM bucket-scoping work without depending on Gen2 entity tagging.

```
# OpenPipeline enrichment example
matcher: matchesValue(k8s.namespace.name, "*prod*")
field: environment
value: prod
```

The `TagTransformer` emits one OpenPipeline enrichment rule per unique NR tag pattern. Its matchers use `contains(entity.name, "...")`, which matching conditions reject (§ Embedded Translation Context), so rewrite them with `matchesValue`. (Gen2 auto-tagging is also supported for hybrid tenants but is not the recommended Gen3 approach.)

5. Cost Optimization Patterns

Migration is a chance to reset the cost profile. Common patterns:

Optimization	Approach
Drop noisy health-check logs	OpenPipeline drop rule (catches 15–25% of log volume)
Drop DEBUG in production	OpenPipeline drop rule scoped to prod namespaces
Tier infra logs to 14 days	Route to a 14-day bucket (vs. 35-day default)
Tier compliance logs to 365 days	Route to a usage-based pricing bucket
Extract metrics from logs	OpenPipeline metric extraction (avoids querying raw logs)
Sample low-value high-volume logs	OpenPipeline sampling processor

Combined, these typically yield **30–60% lower DPS spend** than a default-bucket-only deployment. See **ORGNZ-02 / ORGNZ-99** for the full Grail bucket design pattern.

6. Validation Queries

After log migration, run these DQL queries to confirm:

Volume parity (DT vs. NR)

```
fetch logs, from:-1d
| summarize records = count(), by:{dt.system.bucket}
| sort records desc
```

Compare against NR's `SELECT count(*) FROM Log SINCE 1 day ago` for the same window. Volumes should match within ±5%.

Attribute coverage (before writing a drop rule)

```
fetch logs, from:-1h
| summarize {total = count(), service_name = countIf(isNotNull(service.name)),
k8s_workload = countIf(isNotNull(k8s.workload.name)), process_group =
countIf(isNotNull(dt.entity.process_group))}
```

Shows which attribute can stand in for NR `service`. A drop rule on an attribute that is mostly null matches nothing.

Drop rule effectiveness

```
fetch logs, from:-1h
| filter matchesValue(content, "*health*") and k8s.workload.name == "frontend"
| summarize records = count()
```

Use the **same condition as the rule**. If the drop rule is working, this returns 0. An unscoped check (`contains(content, "health")` alone) also counts health-check lines from sources the rule was never meant to drop, so it cannot reach 0.

Parsing extraction

```
fetch logs, from:-1h
| filter isNotNull(client_ip)
| summarize n = count(), by:{client_ip}
| sort n desc
| limit 10
```

If parsing is working, parsed fields should be non-null.

Tag application (Gen3 — DQL on enriched data)

```
fetch logs, from:-1h
| filter environment == "prod"
| summarize records = count(), by:{environment}
```

Should return enriched data tagged with the expected environment value. For Gen2 entity-tag verification on hybrid tenants, filter the entity's `tags` array:

```
fetch dt.entity.host
| filter in("env:prod", tags)
| fields id, entity.name, tags
```

The string must match the tag exactly as stored, including any `[Context]` prefix (for example `[Environment]dt.cost.product:Product1`). There is no `tag()` function in DQL.

Summary

The logs/tags/drops layer is where migration pays for itself in cost terms. Drop rules port directly; Grok→DPL parsing is mostly mechanical; tags become OpenPipeline enrichment rules that attach attributes to ingested data (Gen3 pattern). Combined with a tiered Grail bucket strategy (see **ORGNZ-02 / ORGNZ-99**), expect 30–60% cost reduction.

Continue to **NRLC-08 Validation, Diff & Rollback** for the verification layer.

Tooling for Logs-Only Migration

> **Check before you rely on the CLI for this layer.** On the tool's `main` branch (read 10/05/2026), `migrate.py migrate` exports and transforms only `dashboards`, `alerts`, `notification_channels`, `synthetics`, `slos` and `workloads`. `log_parsing`, `tags` and `drop_rules` appear in `--list-components`, but `migrate` does not export them — a run with `--components log_parsing,drop_rules,tags` reports each as exported and produces nothing. There is also no `--transform-only` flag, and `logs`, `drops` and `parsing` are not component names. Until the CLI covers this layer, migrate it by hand with the patterns in this notebook.

```
# 1. List the component names your checkout accepts
python3 migrate.py migrate --list-components

# 2. Build each drop rule, parsing rule and enrichment in OpenPipeline from §1–§4,
#    testing every DPL pattern and matching condition against sample data first (§3, §6)
# 3. Reconfigure log forwarders (Filebeat / Fluent Bit / Lambda extension) to point at DT
# 4. Dual-ship for 1–2 weeks; compare volume and confirm drops working (§6)
```

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine (planned future home: the `dynatrace-dma` Dynatrace Migration Assistant organization), and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.